

Project Weaver: Engine Architecture & PRD

Version: 3.2 (The Mechanics & Systems Update) **Product Type:** Just-In-Time Procedural RPG Engine (Backend API)

1. Executive Summary

Project Weaver is a stateless, coordinate-based procedural RPG engine. Unlike traditional RPGs that simulate an entire world simultaneously, Weaver uses a **Just-In-Time (JIT) generation architecture**. When an API client requests a coordinate (X, Y) , the engine uses deterministic hashing to generate a town, populate it with NPCs, and simulate decades of history in milliseconds. By layering a lightweight database (tracking "Deltas") over a deterministic generation sequence, the engine creates the illusion of a massive, persistent, living universe while using near-zero idle server resources.

2. Goals & Objectives

- **Infinite Exploration:** Generate a mathematically infinite amount of playable space without requiring manual level design.
- **Emergent Narrative:** Drive gameplay through deeply simulated NPC relationships and macro-political maneuvers (wars, trade, alliances).
- **Strict Determinism:** Ensure that hitting the same coordinate at the same point in time always yields the exact same world state, artifacts, and NPC IDs.
- **Stateless Client Interaction:** The backend remains completely stateless. The client is responsible for storing and sending the `playerState` (inventory, stats, titles) with every action.

3. Core Architecture: "Seed & Delta"

To achieve infinite scale without infinite storage, the engine does not save generated worlds. It generates them from a mathematical seed on the fly. It only saves the **Deltas** (the persistent changes the player makes, or cross-coordinate events like immigrants and territory claims).

3.1. Universal Time & The JIT Resolution Order

Time is universal and stored in the database under the `GLOBAL` namespace as `globalYear` (default starting at Year 51).

When a client requests `GET /api/chunk/:x/:y`, the engine executes a strict 4-pass pipeline:

1. **The Base Pass:** Generates the town, its biome, and its founding NPCs at Year 1 based on the seed `"world_X_Y"`. Calculates Base Population: $\text{BasePop} = \text{Math.floor}(\text{rng}() * 10) + 10$.
2. **The Legends Pass:** Simulates 50 years of base history. NPCs age, marry, feud, and die.

3. **The Delta Pass:** The engine reads the SQLite database for this coordinate and applies historical overrides. If a player previously stole an item or killed the Mayor, the ECS state is mutated here. *This pass also checks the GLOBAL DB to see if this coordinate has been claimed by a neighboring expanding city.*
4. **The Future Pass:** The engine calculates `globalYear - 51`. If greater than 0, it runs the simulation forward by that many years to catch the chunk up to the present global moment.

3.2. Entity-Component System (ECS)

The engine utilizes `miniplex`. Every actor, town, and child is an entity composed of data components:

| **Component Name** | **Data Payload** | **Purpose** || Identity | `name`, `type`, `id` (UUID) | Defines the entity. UUIDs are deterministic hashes of the seed, year, and name. || Location | `tiles` (array), `parent_id` | Array of X/Y pairs supporting multi-tile cities. || History | `events` (array of strings) | Chronological logs (e.g., "[Year 12] Became Mayor."). || Knowledge | `memories` (dict) | Maps NPC UUIDs to statuses (`loves`, `hates`, `avenged`). || Inventory | `items` (array of objects) | Rich Artifact objects with their own UUIDs and lore. || Status | `state` (enum) | `Alive`, `Dead`, `Migrated`, `Exiled`. || Political | `tier` (int), `demographics` | Tracks settlement size (1-5) and dominant traits (e.g., "Scholar Heavy"). || Diplomacy | `allies`, `colonies`, `suzerain` | Arrays of Coordinate keys mapping macro-level relationships. |

4. NPC Lifecycle & Micro-Simulations

4.1. Demographics, Generation & Replenishment

When a coordinate is generated, its biome deterministically dictates its initial demographic weighting.

Biome Demographic Weightings:

- **Mountain:** 40% Guard, 30% Blacksmith, 10% Bandit, 10% Merchant, 10% Citizen. (*Political Alignment: Militaristic & Industrial*)
- **Forest:** 40% Scholar, 30% Merchant, 10% Guard, 10% Cultist, 10% Citizen. (*Political Alignment: Knowledge & Trade*)
- **Desert:** 40% Bandit, 30% Merchant, 10% Guard, 10% Beggar, 10% Citizen. (*Political Alignment: Scarcity & Opportunism*)
- **Marsh:** 50% Cultist, 20% Beggar, 10% Bandit, 10% Scholar, 10% Citizen. (*Political Alignment: Occult & Isolationist*)
- **Plains (Default):** 60% Citizen, 10% Guard, 10% Merchant, 10% Scholar, 10% Bandit. (*Political Alignment: Balanced*)

Political AI Application: The engine tallies these roles at the end of base generation. The town's governing AI is locked into a macro-political stance based on which groups cross a 50%

threshold (e.g., *Aggressive* if Guards+Bandits > 50%, *Federation* if Scholars+Merchants > 50%).

Population Replenishment Rule: To prevent JIT towns from dying out over long Future Passes (e.g., 200 years of simulation), a check runs every simulated decade. If the living population falls below 5, a "Refugee Crisis" or "Baby Boom" event is triggered, instantly spawning $\text{Math.floor}(\text{rng}() * 5) + 3$ new NPCs matching the biome's demographic weights.

4.2. Migration Tracking & Schrödinger's Immigrant

Migrations are entirely reactive to *loaded* chunks to preserve JIT performance.

- **The Rule:** An immigrant will only arrive at Destination B if Origin A was previously loaded and simulated by a player.
- **Origin Logging:** If family units decide to migrate, their status is set to `Migrated`. A history event is pushed, and tracking metadata is added: `"migrationData": { "destination": { "x": 20, "y": 30 }, "migratedAtYear": X } .`
- **Destination Injection:** The engine writes a JSON Delta to the `GLOBAL` database at the target coordinate (`world_X20_Y30`).
 - **Key:** `immigrant_data`
 - **Payload:** The NPC's stats, UUID, and origin explicitly mapped.
 - When the player eventually visits X:20, Y:30, the Delta Pass pulls this data and injects the NPC.

4.3. Rich Artifact System & Generation Mechanics

Artifacts are crafted dynamically. Every simulated year, Scholars, Blacksmiths, and Cultists have a **5% chance** to generate a new Artifact based on their role and town's Political Stance.

- **Tomes (Knowledge):** Crafted by Scholars.
 - **Mechanic:** Reading high-tier Tomes extracts explicit X/Y coordinates of generated Ruins or Colonies.
 - **Role Shift:** Gifting to a Citizen has a 75% chance to convert them to a Scholar.
- **Jewelry (Corruption):** Discovered by Bandits/Merchants.
 - **Subversion:** Gifting cursed jewelry instantly corrupts an NPC's role (e.g., Guard -> Cultist).
 - **Diplomatic Weight:** Provides a +20% success modifier to `/ally` or `/trade` actions if held by the acting Mayor.
- **Weapons (Elevation):** Crafted by Blacksmiths.
 - **Militarization:** Gifting elevates lower-class NPCs to Guards or Heroes.
 - **War Modifiers:** Grants +5 to a town's Offense/Defense score per Weapon held by a Guard/Hero.
- **Relics (Anomalies):** Discovered by Cultists.

- **Paradigm Shifts:** Dropping Relics triggers massive ECS events (e.g., 50% of the town converts to Cultists, or a spontaneous migration is forced).

5. The Macro-Political Engine & Conquest

Settlements progress through tiers: **Town (1x1)** -> **Small City (2x2)** -> **Full City (3x3)** -> **Magistrate** -> **Kingdom**.

5.1. Territory Claiming (Tile Expansion)

When a town levels up to a Small City, it must claim adjacent tiles.

- **The Claim Rule:** The expanding city writes a Delta to the `GLOBAL` database for the target coordinate (e.g., `Claimed_By: "world_X10_Y15"`).
- **Chunk Generation Override:** If a player walks into a claimed tile, the Base Pass skips generic town generation. Instead, it generates a "District" (e.g., Slums, Market, Keep) that shares the parent city's ruler, diplomacy, and history logs.

5.2. Conflict Resolution (The Math of War)

When expansion limits are reached, Aggressive or Opportunistic towns invade. The combat is resolved strictly mathematically using deterministic peeking:

1. **Offense Score:** $(\text{Invader Guards} * 2) + (\text{Invader Heroes} * 5) + (\text{Invader Weapons} * 5) + (\text{Suzerain Guards} * 0.5)$
2. **Defense Score:** $(\text{Target Guards} * 2) + (\text{Target Heroes} * 5) + (\text{Target Weapons} * 5) + (\text{Allied Guards} * 0.5)$
3. **Resolution Checks:**
 - **Win Margin > 10 (Crushing Victory):** Loser is annexed. Mayor executed. 50% of loser's inventory transferred to winner's NPCs.
 - **Win Margin > 0 (Subjugation):** Loser becomes a Colony. Mayor survives as a Puppet. 1 Artifact per decade is taxed and sent to Suzerain.
 - **Win Margin < 0 (Defeat):** Invasion fails. Invading Mayor loses 50% of their Guard population (marked as Dead).

6. Player Agency & Progression Mechanics

6.1. XP, Leveling, and Stats

The `playerState` maintains core stats that govern the success rates of backend actions.

- **Progression Loop:**
 - * Steal an item: +25 XP
 - Turn in Fetch/Heist Quest: +50 XP
 - Report Bounty: +100 XP
 - Usurp Kingdom (Regicide): +5000 XP

- **Leveling:** Every 100 XP grants 1 Level. Each level allows the client to increment `stealth` or `strength` by 1.

6.2. Action Resolution Checks

- `/api/action/steal` : $\text{FailChance} = \text{Math.max}(0.10, 0.60 - (\text{playerState.stats.stealth} * 0.05))$. Failure drops coordinate reputation by 10 and confiscates a random player artifact.
- `/api/action/assassinate` : $\text{FailChance} = \text{Math.max}(0.10, 0.80 - (\text{playerState.stats.stealth} * 0.05) - (\text{playerState.stats.strength} * 0.05))$. Equipped Weapons grant a hidden +0.10 to success. Failure drops reputation by 50.
- **Regicide (Kingdom Takeover):** Requires targeting an NPC with the `King` role. Base `FailChance` is locked at **0.95**. Requires massive Stealth/Strength stacking and high-tier Weapons to succeed. Success transfers the `Kingdom` title and territory to the player.

6.3. Coordinate-Scoped Reputation

Reputation is tracked strictly via coordinate keys to support changing borders.

```
"playerState": {
  "reputation": {
    "world_X10_Y15": 50,
    "world_X20_Y30": -100
  }
}
```

Note: If `world_X20_Y30` is a Colony, actions there propagate 50% of their reputation changes to the Suzerain's coordinate.

7. Spatial Visibility (Fog of War API)

To allow the frontend to render mini-maps or neighbor tiles without crashing the server by simulating 9 chunks simultaneously, a lightweight metadata endpoint is used.

Endpoint: GET `/api/map/:x/:y/:radius`

7.1. Performance & Execution

- **No ECS Instantiation:** The engine bypasses Miniplex entirely.
- **Math Only:** It runs the coordinate seed through a fast table to determine the biome and base settlement presence.
- **Delta Peeking:** It queries the `GLOBAL` DB for `Claimed_By` tags. If a tile is claimed, it returns the Parent City's metadata instead of generating a generic village.

Sample Tile Payload:

```
{  
  "x": 10,  
  "y": 11,  
  "biome": "Forest",  
  "hasSettlement": true,  
  "settlementTier": 2,  
  "politicalStance": "Hostile",  
}
```